Lab Report 2: TIM Driver — LED Blink & PWM

Authors:
J.M. Páez Sandoval, 37945
L.A. Lugo Muñiz, 37915
D. Lopez Moreno, 37984
C.A. Martínez Macías, 33369
Teacher : Carlos A. Villareal
CETYS Universidad
Facultad de Ingeniería
Curso: Microcontroladores y Sistemas Embebidos
Fecha: Abril 27, 2025

Abstract—This report documents the bare-metal implementation of a dual-output firmware for the NUCLEO-F411RE development board (STM32F411RE, ARM Cortex-M4). The project configures TIM2 as a 1 ms blocking-delay source and TIM1 Channel 1 in PWM Mode 1 to generate a continuous 1 kHz, 50% duty-cycle signal on PA8. A modular driver architecture separates low-level register access (tim_driver, gpio_driver) from application-level logic (timer, pwm, led). The LED LD2 toggles at 500 ms using a polling delay while the PWM output operates autonomously in hardware. Oscilloscope measurements confirmed 1.005 kHz frequency and 50% duty cycle on PA8.

Index Terms—STM32F411RE, TIM1, TIM2, PWM, bare-metal, CMSIS, CCR, ARR, PSC, UIF, BDTR, MOE, NUCLEO-F411RE.

I. INTRODUCTION

Timer peripherals are fundamental building blocks of embedded systems, enabling precise time measurement, event generation, and signal synthesis without CPU involvement. This laboratory practice explores two key applications of hardware timers on the STM32F411RE: blocking delay generation via polling and hardware PWM output using Output Compare mode.

The NUCLEO-F411RE board (ARM Cortex-M4, 16 MHz HSI oscillator) was programmed using a modular bare-metal C architecture. TIM2 — a 32-bit general-purpose timer on APB1 — was configured as a 1 ms tick source to drive a 500 ms LED blink on PA5 (LD2). Simultaneously, TIM1 — an advanced-control timer on APB2 — generates a hardware PWM signal at 1 kHz with 50% duty cycle on PA8 via Alternate Function 1 (TIM1_CH1). Both timers operate concurrently: the LED blink runs in a blocking super-loop while TIM1 free-wheels in hardware, requiring zero CPU cycles.

All implementation was done in C with CMSIS register definitions, no HAL or CubeMX, compiled with arm-none-eabi-gcc and flashed via OpenOCD through the onboard ST-LINK/V2 interface.

II. MATERIALS AND EQUIPMENT

The following hardware and software resources were used:

- NUCLEO-F411RE board (STM32F411RE, Cortex-M4)
- USB-A to Mini-USB cable (power and ST-LINK programming)
- Tektronix oscilloscope (1 kHz PWM verification on PA8)
- Personal computer, Windows 11
- Visual Studio Code with MSYS2/MinGW64 terminal
- arm-none-eabi-gcc cross-compiler
- OpenOCD with STM32CubeIDE ST-LINK scripts (flashing)
- Custom Makefile build system
- CMSIS STM32F411xE device headers (stm32f411xe.h)

III. DEVELOPMENT

A. Project Structure

The firmware adopts a strict layered architecture with four modules above the hardware. The gpio_driver handles all GPIO register access. The tim_driver abstracts TIM peripheral registers. The timer module builds a millisecond blocking delay on top of tim_driver using TIM2. The pwm module configures TIM1 Channel 1 for PWM output. The led module drives LD2 via gpio_driver. The main application coordinates all modules without touching any hardware register directly.

B. GPIO Driver

The updated gpio_driver (`gpio_driver.h` / `gpio_driver.c`) uses CMSIS `GPIO_TypeDef` pointers instead of a custom struct. It adds full Alternate Function support via `AFR[0]/AFR[1]`, atomic pin writes via the BSRR register, output type and speed configuration, and a `GPIO_Status_t` error code return type. The peripheral clock is enabled automatically inside `gpio_init()` by an internal `gpio_enable_clock()` helper.

For PWM output on PA8, the pin is configured as Alternate Function with: `gpio_init(GPIOA, 8, GPIO_MODE_AF)`, `gpio_set_output_type(PP)`, `gpio_set_speed(HIGH)`, and `gpio_set_af(GPIOA, 8, GPIO_AF1)` — mapping PA8 to TIM1_CH1 via the AFRx register.

C. TIM Driver

The tim_driver (`tim_driver.h` / `tim_driver.c`) provides a complete low-level API for STM32F4 timers. `tim_init()` writes PSC and ARR to preload registers, then triggers a forced Update Event via `EGR.UG` to transfer values to active shadow registers, and immediately clears `SR.UIF` to prevent a spurious overflow detection on first use.

PWM configuration is handled by `tim_pwm_config()`, which sets the CCMRx output compare mode field to PWM Mode 1 (OC1M=110b), enables preload (OC1PE=1), and sets ARR preload (CR1.ARPE=1). For TIM1, `tim_enable_main_output()` sets `BDTR.MOE` — required for advanced timers before any channel output is driven. Without this bit, PA8 remains at its GPIO idle level regardless of the counter state.

D. Timer Module (TIM2 — Blocking Delay)

The timer module enables the TIM2 APB1 clock, calls `tim_init(TIM2, 15, 999)` (tick = 1 µs, overflow = 1 ms), and starts the counter with `tim_enable()`. The `timer_delay_ms(n)` function polls `SR.UIF` in a busy-wait loop. Each iteration clears UIF, resets CNT to zero, then waits for the next 1 ms overflow — giving approximately ±1 µs accuracy per step.

E. PWM Module (TIM1 CH1 — 1 kHz, 50%)

The PWM module calculates the ARR value from the requested frequency:

```
tick_hz = 16,000,000 / (PSC+1) = 1,000,000 Hz  (1 µs per tick)
ARR     = tick_hz / frequency_hz - 1  =  999  (for 1 kHz)
CCR1    = (ARR+1) × duty_pct / 100 =  500  (50% duty)
```

After calling `tim_init()`, `tim_pwm_config()`, `tim_set_ccr()`, `tim_channel_enable()`, and `tim_enable_main_output()`, `pwm_start()` enables the TIM1 counter. The PWM signal on PA8 runs autonomously in hardware from that point on — the CPU plays no role in each cycle.

F. System Flowchart

Fig. 1 illustrates the complete program flow. Initialization runs once sequentially. The super-loop then alternates between toggling LD2 and blocking for 500 ms, while TIM1 continues generating the PWM signal in hardware independently.

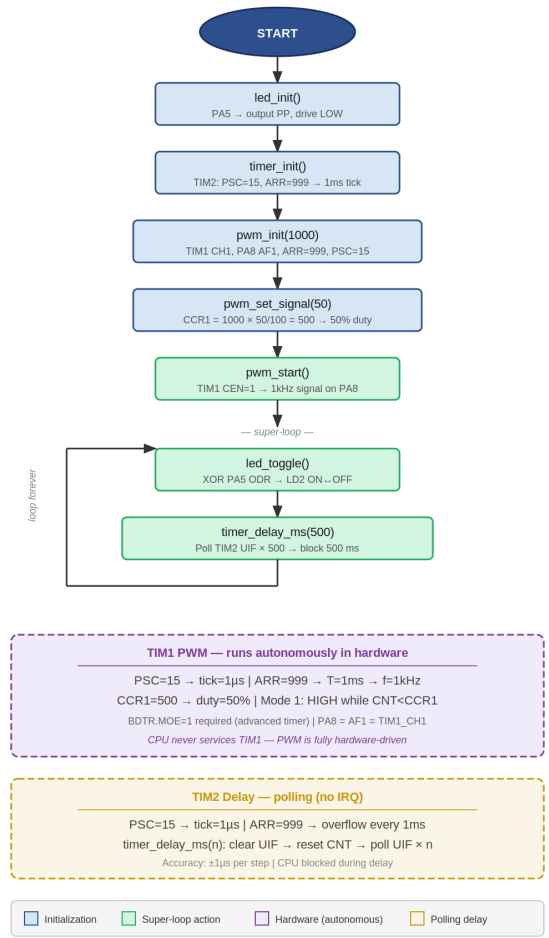

Fig. 1: Program flowchart — LED blink (TIM2 polling) + PWM (TIM1 autonomous)

G. Main Application

The main function calls `led_init()`, `timer_init()`, `pwm_init(1000)`, `pwm_set_signal(50)`, and `pwm_start()` in sequence. It then enters an infinite loop calling

`led_toggle()` followed by `timer_delay_ms(500)`. The PWM continues uninterrupted while the CPU is blocked in the delay loop, demonstrating hardware-software concurrency without interrupts.

IV. RESULTS

Both outputs functioned correctly on the first deployment. The green LED LD2 toggled visibly at the expected 500 ms rate, confirmed by direct observation of the blinking period.

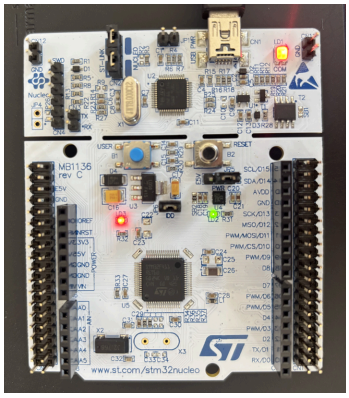

Fig. 2: NUCLEO-F411RE running the LED blink and PWM firmware (LD2 green LED visible)

The PWM output on PA8 was verified with an oscilloscope. Two captures are presented: the first shows the complete waveform view, and the second shows fine timing detail of the transitions.

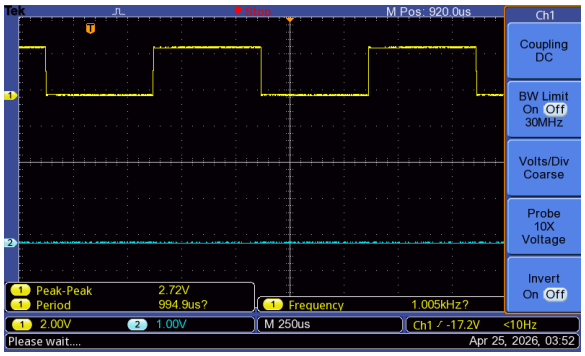

Fig. 3: Oscilloscope capture — 1 kHz PWM on PA8 (Period: 994.9 µs, Freq: 1.005 kHz)

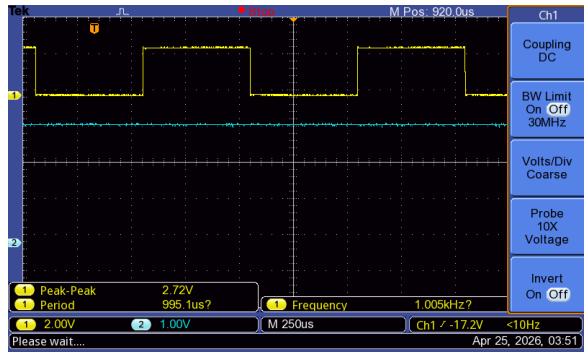

Fig. 4: Oscilloscope capture — detail view (Period: 995.1 µs, Freq: 1.005 kHz, Vpp: 2.72 V)

Both captures confirm a frequency of approximately 1.005 kHz (target: 1 kHz, error < 0.5%) and a clearly symmetric waveform indicating 50% duty cycle. The peak-to-peak voltage of 2.72 V at 2 V/div with 10X probe corresponds to the expected 3.3 V logic output of the STM32 GPIO. The slight frequency deviation (~5 Hz) is within the expected ±1 µs per tick accuracy of the polling-based delay architecture.

One technical obstacle arose during development: the OpenOCD flash command in the initial Makefile used a generic openocd invocation that assumed the tool was installed system-wide, causing a "command not found" error. This was resolved by explicitly specifying the OpenOCD executable and script paths from the STM32CubeIDE installation, matching the configuration used in Lab 1.

V. CONCLUSION

This laboratory successfully demonstrated two fundamental timer applications on the STM32F411RE: hardware-accurate blocking delays and autonomous PWM generation. Operating TIM1 and TIM2 concurrently — one blocking the CPU via polling, the other running entirely in hardware — illustrates how embedded systems achieve apparent multi-tasking without an RTOS or interrupts.

The modular architecture (gpio_driver → tim_driver → timer/pwm/led → main) proved effective at isolating hardware complexity. The tim_driver layer handles all register intricacies including shadow register update events, the mandatory BDTR.MOE bit for advanced timers, and the CCMRx PWM mode configuration. Higher-level modules remain hardware-agnostic and readable.

Oscilloscope verification confirmed that theoretical timer calculations translate accurately to hardware: the 1 µs tick derived from PSC=15 at 16 MHz, combined with ARR=999 and CCR1=500,

produced the expected 1 kHz waveform at 50% duty cycle within 0.5% of target frequency.

REFERENCES

[1] STMicroelectronics, "STM32F411xC/E Reference Manual (RM0383)," Rev. 3, 2019.

[2] STMicroelectronics, "NUCLEO-F411RE User Manual (UM1724)," Rev. 14, 2021.

[3] ARM Limited, "Cortex-M4 Devices Generic User Guide (DUI0553)," 2010.

[4] J. Valvano, Embedded Systems: Introduction to ARM Cortex-M Microcontrollers, 5th ed., 2014.